

1. Order Management (Quản lý Đơn hàng)

Mục tiêu (Core Logic): Quản lý Master Data của Đơn hàng. Là đầu vào khởi tạo toàn bộ chuỗi sự kiện trong hệ thống

Cấu trúc Database (1-N):

order_cutsheets (Bảng cha): Lưu thông tin chung (PO, Mã khách, Ngày giao).

order_sizes (Bảng con): Rã chi tiết số lượng theo Size (S, M, L...). Tránh thiết kế Size theo dạng cột ngang (Hard-c

Logic cần chú ý (Edge Cases):

Trạng thái (Status): Nếu Đơn hàng chuyển sang trạng thái In Production (Đang sản xuất) hoặc Closed, phải **khóa**

Giao dịch (DB Transaction): Khi tạo Đơn hàng, việc lưu vào bảng cutsheets và sizes phải nằm trong một Transact

2. BOM & Tech Pack (Định mức & Kỹ thuật)

Mục tiêu (Core Logic): Lưu trữ cấu trúc cây sản phẩm và công thức tính vật tư.

Cấu trúc Database:

styles -> bom_items -> bom_colorways.

Cần có bảng Mapping Master Data: materials (Mã vật tư nội bộ) và material_vendors (Mã vật tư của Nhà cung cấp

Logic cần chú ý (Edge Cases):

Tính năng Clone (Copy): Dev cần viết một API cho phép "Clone BOM" từ Mã áo A sang Mã áo B, vì các mã áo thườ

3. Material Requirements Planning - MRP (Hoạch định Vật tư)

Mục tiêu (Core Logic): Module tính toán nặng nhất hệ thống. Xử lý bài toán: *Nhu cầu (Gross) - Tồn kho khả dụng (A*

Logic cốt lõi (Core Algorithm):

Hệ thống không lấy On-hand Inventory (Tồn thực tế) để trừ, mà phải dùng Available Inventory (Tồn khả dụng).

Công thức: Available = On-hand - Allocated (Hàng đã giữ chỗ cho lệnh cắt khác) + On-Order (Hàng đang đi đư

API & Tối ưu hóa:

Vì vòng lặp tính MRP rất lớn, không nên chạy Real-time mỗi khi user load trang. Nên thiết kế chạy theo dạng **Backg**

Kết quả tính được lưu vào bảng tính mrp_suggestions để giao diện query cho nhanh.

4. Procurement & Sourcing (Mua hàng)

Mục tiêu (Core Logic): Chuyển đổi Đề xuất mua (MRP Suggestions) thành Đơn mua hàng (PO). Theo dõi lịch hàng

Cấu trúc Database (1-N):

purchase_orders (Bảng cha) -> po_items (Bảng con).

Logic cần chú ý (Edge Cases):

Cho phép gom nhiều dòng mrp_suggestions của cùng 1 Nhà cung cấp vào 1 mã purchase_order duy nhất.

Dữ liệu cột eta_date (Ngày hàng về) trong bảng po_items phải được API đẩy ngầm (trigger/webhook) sang module

5. Inventory / Warehouse (Quản lý Kho hàng)

Mục tiêu (Core Logic): Quản lý số dư vật lý theo Lô/Cuộn và lịch sử biến động.

Cấu trúc Database (Nguyên tắc ACID):

inventory_balances (Bảng chốt số dư hiện tại).

inventory_transactions (Bảng ghi log biến động Nhập/Xuất/Điều chỉnh).

goods_receipts & goods_receipt_items (Phiếu nhập từ PO).

Logic cần chú ý (Edge Cases):

Concurrency (Tranh chấp dữ liệu): Khi 2 user cùng xuất kho 1 cuộn vải cùng lúc, Dev phải dùng cơ chế khóa (Row

Tính toàn vẹn (Integrity): KHÔNG bao giờ cho phép người dùng sửa tay (Update) vào cột balance_qty. Mọi biến đ

PHÂN HỆ LỆNH SOẠN HÀNG & XUẤT KHO SẢN XUẤT

MỤC TIÊU (CORE LOGIC & OBJECTIVES)

Số hóa tác nghiệp kho: Thay thế phiếu yêu cầu vật tư bằng giấy. Chuyển đổi dữ liệu Đơn hàng (Cut Sheet - CS) và E

Kiểm soát xuất kho: Đảm bảo kho xuất đúng mã vật tư, đúng màu, đúng số lượng và trừ đúng tồn kho của cuộn/lô (Cầu nối dữ liệu (Cross-module Bridge): Là điểm giao tiếp giữa Kế hoạch (người ra lệnh), Kỹ thuật (người ra công t

CẤU TRÚC DATABASE (DATABASE SCHEMA)

Phân hệ này yêu cầu thiết lập 4 bảng kết nối theo mô hình 1-N (Một - Nhiều) và liên kết chặt chẽ với bảng Tồn kho (in **Nhóm Lệnh Yêu Cầu (Requisition):**

material_requisitions (Bảng cha): Lưu thông tin tổng quan của lệnh. Bắt buộc có cutsheet_id (Khóa ngoại) để biết requisition_items (Bảng con): Lưu chi tiết vật tư cần xuất. Có cột requested_qty (Số lượng yêu cầu, do hệ thống tự

Nhóm Phiếu Thực Xuất (Issue):

material_issues (Bảng cha): Lưu thông tin phiếu xuất kho tác nghiệp (Mã phiếu, ngày xuất, người xuất). Có khóa n issue_items (Bảng con): Lưu chi tiết từng cuộn/lô hàng thực tế được bốc khỏi kệ. Bắt buộc chứa lot_roll_no (Số lô

3. LƯU Ý CHO DEV (EDGE CASES & CONSTRAINTS)

Để hệ thống không bị lỗi dữ liệu (Data Inconsistency) và thất thoát hàng hóa, Dev cần đặc biệt chú ý xử lý các vấn đề

Tính Toàn Vẹn Của Giao Dịch (ACID Transactions): Khi user bấm "Chốt Phiếu Xuất" (Post Issue), Backend **bắt bu**

- Trừ số lượng ở bảng tồn kho (inventory_balances).
- Ghi log lịch sử (inventory_transactions).
- Cộng dồn số lượng đã xuất vào cột issued_qty của bảng requisition_items. *(Nếu lỗi ở bất kỳ bước nào, phải Roll*

Tranh Chấp Dữ Liệu (Concurrency Control): Khi query bảng tồn kho để trừ số lượng, phải dùng cơ chế khóa dòng (

Xử Lý Dung Sai Xuất Lỗi (Over-issue Tolerance): Ngành may không thể cắt lẻ 5 mét vải rồi cắt lại kho. Dev phải ch

Tự Động Hóa (Automation Triggers):

Trigger Khởi tạo: Khi Đơn hàng (Cut Sheet) được đổi status sang RELEASED (Phát lệnh), Backend cần tạo một Jc

Trigger Hoàn thành: Khi cột issued_qty >= requested_qty của tất cả vật tư, hệ thống tự động đổi status của lệnh y

6. Master Production Schedule - MPS (Kế hoạch Sản xuất)

Mục tiêu (Core Logic): Quản lý dữ liệu chuỗi thời gian (Time-series) để xếp lịch cho máy/chuyên may.

Cấu trúc Database:

work_orders (Lệnh sản xuất) -> mps_schedules (Phân bổ chuyền, ngày bắt đầu, ngày kết thúc).

Logic cần chú ý (Edge Cases):

Validation: Ngày Planned Start Date bắt buộc phải lùi sau ngày ETA Date của vật tư cuối cùng về kho. Nếu không

Một Lệnh sản xuất (Work Order) có thể được tách ra chạy song song trên nhiều Chuyền (Sewing Lines) khác nhau

7. Shop Floor Control - SFC (Kiểm soát Phân xưởng)

Mục tiêu (Core Logic): Nhập liệu sản lượng hàng ngày với tốc độ cao, tính toán WIP (Bán thành phẩm).

Cấu trúc Database:

daily_production_logs -> Nối với mps_schedules để biết sản lượng thuộc Kế hoạch nào.

Logic cần chú ý (Edge Cases):

WIP Calculation Logic: $WIP_Sewing = Total_Cut_Qty - Total_Sew_Qty$.

Chặn Over-production: API phải check số lượng actual_qty nhập vào không được phép vượt quá số lượng của cô

8. Finance & Costing (Tài chính & Giá thành)

Mục tiêu (Core Logic): Tổng hợp dữ liệu (Aggregation & JOINS) từ Kho, Sản xuất, Mua hàng để ra báo cáo Lãi/Lỗ.

Logic cần chú ý (Edge Cases):

Snapshot Data: Khi một Đơn hàng Closed, chi phí (vật tư, nhân công) của đơn hàng đó phải được sao chép (snap

Lý do: Nếu Dev chỉ viết API query JOIN lấy giá vật tư hiện tại, thì 1 năm sau xem lại báo cáo, giá vật tư thay đổi sẽ

ground Job / CRON Job (ví dụ 12h đêm chạy 1 lần) hoặc có một nút [Run MRP] để user chủ động trigger.

ộng phải thông qua việc Insert một dòng vào inventory_transactions. DB Trigger hoặc Application Logic sẽ tự động cộ

ước) và Kho (người thực thi). Cung cấp trạng thái vật tư (Material Readiness) theo thời gian thực cho Phân xưởng sản

lệnh này soạn cho mã CS nào. Các trạng thái: PENDING (Chờ soạn), PARTIAL (Đang xuất dở dang), COMPLETED (Đã

l/cuộn), location_id (Vị trí kệ), và issued_qty (Số lượng thực xuất). Cần có khóa ngoại req_item_id để map đúng với d

(VD: SELECT ... FOR UPDATE trong SQL/TypeORM) để ngăn chặn lỗi 2 người cùng xuất 1 cuộn vải trong cùng 1 giây
phép issued_qty (thực xuất) lớn hơn requested_qty (yêu cầu). Tuy nhiên, cần thiết lập biến môi trường (Environmen

chạy ngầm để tự động tính toán BOM và Insert dữ liệu vào bảng material_requisitions. Không bắt user nhập tay lại c
yêu cầu thành COMPLETED và gửi Webhook/Notification sang module Shop Floor (Phân xưởng) báo hiệu "Vải đã sẵn

t Variable) ví dụ MAX_TOLERANCE = 5%. Nếu xuất lỗi dưới 5% thì cho qua, vượt quá 5% thì Backend block lại và trê

ả về lỗi HTTP 400.